



音乐播放器 · 答辩要求

答辩全程 3 ~ 6 分钟，图文并茂地演示你的作品，分两步进行。

第一步：功能演示（必做，对照要求逐条过）

这一步是答辩的硬性环节——必须现场把程序跑起来、逐条演示，否则无法证明这些功能是你真正做出来的。

- 打开必做功能清单（9 项），**一条一条现场演示**你的程序实现了什么、效果如何。
- 完成的：直接演示效果；**没做完或做得不完善的，要如实说明**——做到哪一步、卡在什么地方。

如实说明比假装完成好；隐瞒未完成项、演示时“翻车”又解释不清，反而更扣印象分。

- 做了的**加分项**（歌词特效、频谱可视化、MV、换肤、自建歌单等）一并展示。

第二步：介绍你的设计思路

下面这些方面，**在你的设计文档里都要写到**（设计文档是评分的重要依据）。而答辩口头只有三四分钟、讲不完，所以现场**挑着讲**：优先讲【优先讲】的两块，时间够再补其余。

优先讲（这两块一定要讲到）

（一）面向对象设计

- **类怎么拆的**：你把程序分成了哪几个类、每个类负责什么，为什么这么分（单一职责——一个类只管一件事）。
- **多态用在哪**：哪里用了“同一个基类/接口、多种实现”（当某个行为有好几种不同做法时），带来了什么好处。
- **封装与解耦**：把某个框架/库的类包一层你自己的接口，让界面依赖你的抽象、而不是直接依赖它（隔离变化）。
- **组合关系**：用“一个对象由若干更小的对象组成”来组织代码。

- **界面与数据分离、用信号槽降低模块耦合。**
- **可扩展性自评：**现在再加一种播放模式 / 一种歌词特效，要改动多少代码？（开闭原则）

（二）与 AI 的协作

- 用了哪个 / 哪些 **AI 工具或模型**（如 ChatGPT、Claude、GitHub Copilot、通义、DeepSeek 等），分别拿来做什么。
- 你怎么把任务**拆开**喂给 AI 的；哪些提示词有效、哪些没用。
- **AI 给出错误 / 不能编译的代码时，你怎么发现并纠正的；**哪些是你自己写 / 改 / 重构的。（能体现你是否真的掌控了自己的代码）

其余方面（设计文档里都要写到；口头时间够再补）

- **工程与代码组织：**目录结构怎么划分、`CMakeLists.txt` 怎么组织；命名 / 注释 / 代码风格。
- **关键技术实现与取舍：**歌词同步/拖动怎么做的；转码怎么实现、为什么这么选（对比你考虑过的方案）；持久化用什么格式、存哪、为什么。
- **健壮性与性能：**界面为什么**不卡**（哪些耗时操作放到了异步/后台）；异常情况（歌词缺失、`.lrc` 编码不对、文件损坏、路径含中文等）怎么处理。
- **UI / 交互设计：**布局、配色 / QSS、交互（歌词拖动、进度条、播放模式切换）为什么这么设计。
- **踩过的坑 & 调试：**印象最深的一两个 bug，怎么定位、怎么解决的。
- **亮点 / 创新：**你额外做的、觉得得意的地方。

呈现形式

图文并茂。答辩时用来演示的材料，可以是 **PPT**，也可以直接用你的**设计文档**（Markdown / Word / PDF，含截图）。

但无论用哪种演示，**设计文档都必须提交**（见最后一节）；纯口述、没有任何图文材料，不合格。

时长

3~6 分钟。请提前掐好时间、精简内容，别超时。

提交（务必按时，重要）

下一次上课前至少 2 小时，把下列材料**打包成一个 zip** 提交到系统，文件名用**学号 + 姓名**命名。助教会提前把大家的材料拷到**教室电脑**上准备好。

- 命名格式：学号-姓名.zip，例如 525XXXXXXXX-张三.zip。
- 建议解压后的**顶层文件夹用同样的名字**，避免多人解压后文件混到一起。

1. **设计文档（必交，图文并茂）**；
2. **演示所用的材料**（PPT / 截图 / 录屏等）；
3. **可以直接编译的源代码 + CMakeLists.txt**（助教需能在教室电脑上一键构建运行）。

逾期不交 = 助教无法提前在教室电脑上帮你准备好 = 无法正常答辩。请务必提前！